COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	08/04/26	**DATE SUBMITTED:**	08/11/26
NAME:	Aguila, Daniel C.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA 5.1 QUEUES

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A CODE

ILOA > main.cpp > main()

```cpp
//Aguila, Daniel C. CPE21S1
#include <iostream>
#include <queue>

void display(std::queue<char> r);

int main(){

    std::queue<char> q;

    q.push('c');
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('1');
    q.push('0');

    //display
    display(q);
    //empty()
    std::cout<<"Is the queue empty? "<<q.empty() << std::endl;
    //size()
    std::cout<<"What is the size? "<<q.size() << std::endl;
    //front
    std::cout<<"The front of this queue is: "<<q.front() << std::endl;
    //back
    std::cout<<"The back of this queue is: "<<q.back() << std::endl;
    //pop
    std::cout<<"Popping out: "<<q.front() << std::endl;
```

```
28      //pop
29      std::cout<<"Popping out: "<<q.front() << std::endl;
30      q.pop();
31      //display
32      display(q);
33      //push another element
34      q.push('o');
35      std::cout<<"After pushing an element: "<< std::endl;
36      //display
37      display(q);
38  }
39
40  void display(std::queue<char> r){
41      //create a copy of the queue
42      std::queue<char> c = r;
43
44      while(!c.empty()){
45          std::cout<<" "<<c.front();
46          c.pop();
47      }
48      std::cout<<"\n";
49
50  }
```

ILO A OUTPUT

```
PS C:\Users\TIPQC\Documents\HOA5p1\ILOA> cd "c:\Users\TIPQC\Documents\HOA5p1\ILOA\"
 c p e 0 1 0
Is the queue empty? 0
What is the size? 6
The front of this queue is: c
The back of this queue is: 0
Popping out: c
 p e 0 1 0
After pushing an element:
 p e 0 1 0 o
PS C:\Users\TIPQC\Documents\HOA5p1\ILOA>
```

ILO A ANALYSIS

I have included a queue in my program comprising the elements 'c', 'p', 'e', '0', '1', and '0' using the function push(). This places a new element on the back side of the queue. My display() function is responsible for copying the queue and displaying each element without disturbing the original queue. Then I used the `empty()` function to determine whether there are any elements in the queue. Also, I used `size()` to display how many elements are present in the queue. Using `front()` and `back()`, I identified the front and back elements in the queue respectively. I followed this by performing `pop()` in order to remove the front element of the queue. Lastly, I included an additional element `'o'` to the back of the queue.

ILO B CODE

main.cpp

ILOB > main.cpp > main()

```cpp
#include <iostream>
#include "queuell.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue(&front, &back, 'D');
    std::cout<< front->data << " "<<back->data<<std::endl;
    enqueue(&front, &back, 'A');
    std::cout<< front->data << " "<<back->data<<std::endl;
    enqueue(&front, &back, 'N');
    std::cout<< front->data << " "<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<< front->data << " "<<back->data<<std::endl;
    return 0;
}
```

queuell.h

ILOB > queuell.h > enqueue<T>(qNode<T>**, qNode<T>**, T)

```cpp
//Aguila, Daniel C. CPE21S1
#ifndef QUEUELL_H
#define QUEUELL_H

template <typename T>
```

```cpp
template <typename T>
class qNode {
    public:
    T data;
    qNode *next;
};

//creating a new node
template <typename T>
qNode<T>* new_node(T newData) {
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

template <typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData) {
    qNode<T>* newNode = new_node(newData);

    //inserting to an empty queue
    if((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }
        //inserting an item into a non-empty queue
        //point the backPtr next to the newNode
        (*backPtr)->next = newNode;
        (*backPtr) = newNode;
    }

template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
    //create a temporary node to store the node to be deleted
    if((*frontPtr) == nullptr) {
        std::cout << "Queue is empty." << std::endl;
        return;
    }

    qNode<T>* deleteNode = (*frontPtr);

    //check if the queue has only one node
    if((*frontPtr) == (*backPtr)) {
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
```

```
49              (*backPtr) = nullptr;
50          }
51              (*frontPtr) = deleteNode->next;
52
53          delete deleteNode;
54      }
55
56      //display all elements in the list
57      template <typename T>
58      void display(qNode<T>* frontPtr) {
59          if(frontPtr == nullptr) {
60              std::cout << "Queue is empty." << std::endl;
61              return;
62          }
63          qNode<T>* temp = frontPtr;
64          std::cout << "Queue elements: ";
65          while(temp != nullptr) {
66              std::cout << temp->data << " ";
67              temp = temp->next;
68          }
69          std::cout << std::endl;
70      }
71
72      //return the front variable
73      template <typename T>
74      T getFront(qNode<T>* frontPtr) {
75          if(frontPtr == nullptr) {
76              throw std::runtime_error("Queue is empty.");
77          }
78          return frontPtr->data;
79      }
80
81      #endif
```

ILO B OUTPUT

```
PS C:\Users\TIPQC\Documents\HOA5p1\ILOB> cd "c:\Users\TIPQC\Documents\HOA5p1\ILOB\" ; i
D D
D A
D N
Queue elements: D A N
Front element: D
A N
Queue elements: A N
PS C:\Users\TIPQC\Documents\HOA5p1\ILOB>
```

ILO B ANALYSIS

To construct a queue using linked nodes with templates, I applied several functions and tested the key methods for them. First, I began with enqueue(), which allows adding each character to the queue. It is clear that the front is always in the first place and the back goes to the last inserted element. After adding such elements as D, A and N, the queue shows them in the same order. Then, I performed dequeue(), which makes the queue to delete the front node and shift the queue to the next node, making A the new front. To provide more understanding of the process, I used the display() function, which displays all the elements, as well as the getFront() method.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

CODE

Main.cpp

Welcome | main.cpp | suppqueue.h

main.cpp > main()

```cpp
//Aguila, Daniel C. CPE21S1
#include <iostream>
#include "suppQueue.h"

int main(){
    Printer myPrinter;

    std::cout << "Adding jobs to the queue..." << std::endl;
    myPrinter.addJob(Job(1, "Daniel Aguila", 5));
    myPrinter.addJob(Job(2, "Megan Lee", 10));

    std::cout <<"\nProcessing jobs in the queue..." << std::endl;
    myPrinter.processJob();

    std::cout <<"\n Adding more jobs to the queue..." << std::endl;
    myPrinter.addJob(Job(3, "Peter Parker", 15));
    myPrinter.addJob(Job(4, "Dr Doom", 20));

    std::cout <<"\nProcessing remaining jobs in the queue..." << std::endl;
    myPrinter.processAllJobs();

    std::cout <<"\nProcessing remaining jobs in the queue..." << std::endl;
    myPrinter.processAllJobs();


    return 0;
}
```

Queue.h

suppqueue.h > ...

```
1   //Aguila, Daniel C. CPE21S1
2   #ifndef SUPPQUEUE_H
3   #define SUPPQUEUE_H
4
5   #include <iostream>
6   #include <string>
7
8   class Job{
9   public:
10      int jobID;
11      std::string idName;
12      int pages;
13
14      Job(){
15          jobID = 0;
16          idName = "";
17          pages = 0;
18      }
19      Job(int i, std::string n, int p){
20          jobID = i;
21          idName = n;
22          pages = p;
23      }
24  };
25
26  //create a class the node
27  template <typename T>
28  class qNode{
29      public:
30          T data;
31          qNode<T> *next;
32  };
33
```

```
30          T data;
31          qNode<T> *next;
32      };
33
34      //creates the new node
35      template <typename T>
36      qNode<T>* new_node(T newData){
37          qNode<T>* newNode = new qNode<T>;
38          newNode->data = newData;
39          newNode->next = nullptr;
40          return newNode;
41      }
42
43      //enqueue operation
44      template <typename T>
45      void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
46          //create the new node
47          qNode<T>* newNode = new_node(newData);
48
49          //inserting into an empty queue
50          if((*frontPtr) == nullptr && (*backPtr) == nullptr){
51              (*frontPtr) = newNode;
52              (*backPtr) = newNode;
53              return;
54          }
55          //inserting an item into a non empty queue
56          (*backPtr)->next = newNode;
```

```
56      (*backPtr)->next = newNode;
57      (*backPtr) = newNode;
58  }
59
60  //dequeue
61  template <typename T>
62  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
63      //create a temporary node
64      qNode<T>* deleteNode = nullptr; // temp variable
65      deleteNode = (*frontPtr);
66
67      //check if the queue has only 1 node
68      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
69          (*frontPtr) = nullptr;
70          (*backPtr) = nullptr;
71          delete deleteNode;
72          return;
73      }
74      //delete the node
75      (*frontPtr) = (*frontPtr)->next;
76      deleteNode->next = nullptr;
77      delete deleteNode;
78  }
```

```cpp
template <typename T>
bool isEmpty(qNode<T>** frontPtr){
    return (*frontPtr) == nullptr;
}


class Printer{
private:
    qNode<Job>* front;
    qNode<Job>* back;

public:
    Printer(){
        front = nullptr;
        back = nullptr;
    }

    void addJob(Job newJob){
        enqueue(newJob, &front, &back);
        std::cout << "Job added to the queue: " << newJob.idName << std::endl;
        std::cout << "Job ID: " << newJob.jobID << std::endl;
        std::cout << "Pages: " << newJob.pages << std::endl;
    }

    void processJob(){
        if(isEmpty(&front)){
            std::cout << "No jobs in the queue to process." << std::endl;
            return;
        }
        Job jobToProcess = front->data;
        std::cout << "Processing job: " << jobToProcess.idName << std::endl;
        std::cout << "Job ID: " << jobToProcess.jobID << std::endl;
        std::cout << "Pages: " << jobToProcess.pages << std::endl;
        std::cout << "Pages: " << jobToProcess.pages << std::endl;
        dequeue(&front, &back);
    }

    void processAllJobs(){
        if(isEmpty(&front)){
            std::cout << "No jobs in the queue to process." << std::endl;
            return;
        }
        while(!isEmpty(&front)){
            processJob();
        }
    }

};

#endif
```

OUTPUT

```
[Running] cd "c:\Users\daniel\Downloads\HOA5.1\" && g++ main.cpp -o main && "c:\Users\daniel\Downloads\HOA5.1\"main
Adding jobs to the queue...
Job added to the queue: Daniel Aguila
Job ID: 1
Pages: 5
Job added to the queue: Megan Lee
Job ID: 2
Pages: 10

Processing jobs in the queue...
Processing job: Daniel Aguila
Job ID: 1
Pages: 5

 Adding more jobs to the queue...
Job added to the queue: Peter Parker
Job ID: 3
Pages: 15
Job added to the queue: Dr Doom
Job ID: 4
Pages: 20

Processing remaining jobs in the queue...
Processing job: Megan Lee
Job ID: 2
Pages: 10
Processing job: Peter Parker
Job ID: 3
Pages: 15
Processing job: Dr Doom
Job ID: 4
Pages: 20

Processing remaining jobs in the queue...
No jobs in the queue to process.
```

ANALYSIS

In this supplementary activty, I created a queue of printers where each print request is presented by the `Job` class which has an ID, name of the user who sends it, and the quantity of pages. The class `Printer` performs operations on such jobs using the following methods: `addJob()` adds a new print job to the end of the queue and prints out its characteristics, `processJob()` verifies whether the queue is empty, retrieves a print job from the head of the queue, prints out its properties, and then deletes it via `dequeue()`, while `processAllJobs()` processes print jobs through repeated use of `processJob()`. First, jobs are added, then processed, then new jobs are added and processed, so that the output demonstrates the principle of queuing in action.

The output appeared the way it did because the queue strictly follows the first-in, first-out principle: the earliest job added is always processed first, and later jobs wait their turn. This explains why the program showed jobs being added, processed one by one, and then cleared in the exact sequence they were received.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.